

Федеральное государственное автономное образовательное учреждение высшего
образования

«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Факультет Информационных технологий

Кафедра «Информатики и информационных технологий»

Направление подготовки/ специальность: 09.03.02 Информационные системы и технологии

ОТЧЕТ

по проектной практике

Студент: Жилин Константин Владимирович, Беджанов Азамат Асланович

Группа: 251-333

Место прохождения практики: Московский Политех, кафедра Информационных
технологий

Отчет принят с оценкой _____ Дата _____

Руководитель практики: Семёнова Валерия Валериевна

Москва 2026

ВВЕДЕНИЕ

Настоящий отчёт составлен по результатам прохождения проектной (учебной) практики 2026 учебного года на базе Московского политехнического университета в период с 3 февраля по 24 мая 2026 года.

В рамках практики студенты участвовали в проекте «Развитие дополнительного профессионального образования (ДПО)» Московского политехнического университета. Команда студентов группы 251-333 отвечала за создание коротких рекламных видеороликов о курсах ДПО.

Ссылки на проект:

Репозиторий: <https://github.com/zhilinka/project-practice-2026>

Сайт: <https://zhilinka.github.io/project-practice-2026/>

Тг - бот: @mospolytech_currency_bot

1. Общая информация о проекте

Название проекта: «Развитие дополнительного профессионального образования (ДПО) Московского политехнического университета»

Цель: повысить узнаваемость и привлекательность программ ДПО с помощью современных инструментов digital-продвижения.

Задачи проекта (все команды):

- Изучить целевую аудиторию программ ДПО
- Разработать рекламные текстовые материалы о курсах
- Создать короткие видеоролики, продвигающие курсы ДПО
- Разработать Telegram-бот для информирования о программах ДПО
- Задokumentировать результаты и оформить сайт практики

2. Общая характеристика организации

Наименование заказчика: Федеральное государственное автономное образовательное учреждение высшего образования «Московский политехнический университет».

Московский Политех — ведущий технический вуз Москвы. В структуру университета входит Институт дополнительного профессионального образования, предлагающий программы переподготовки и повышения квалификации.

3. Описание задания по практике

Базовая часть:

1. Настройка Git и репозитория:

- Создать групповой репозиторий на GitHub
- Освоить базовые команды Git: clone, add, commit, push, branch
- Регулярно фиксировать изменения с осмысленными сообщениями

2. Написание документов в Markdown:

- Оформить все материалы проекта в формате Markdown
- Изучить синтаксис Markdown

3. Создание статического веб-сайта:

- Создать сайт на Hugo и опубликовать на GitHub Pages
- Сайт должен включать: Главную, О проекте, Участников, Журнал, Ресурсы
- Оформить страницы графическими материалами

4. Взаимодействие с организацией-партнёром:

- Организовать взаимодействие через куратора проекта
- Написать отчёт в формате Markdown

Вариативная часть:

Практическая реализация технологии — разработка Telegram-бота для конвертации валют на Python с использованием библиотеки python-telegram-bot.

4. Описание достигнутых результатов

4.1 Базовая часть

1. Репозиторий на GitHub

Создан групповой репозиторий project-practice-2026 с правильной структурой папок: docs, site, src, reports. Освоены команды Git.

Рисунок 1 — Главная страница репозитория:

zhilinka

final-report.md

✓

e1255b4 · 36 minutes ago

86 Commits

.github/workflows

Fix Hugo workflow

last week

docs

final-report.md

36 minutes ago

reports

Add files via upload

2 days ago

site

Смена на вариативную часть

48 minutes ago

src

Add files via upload

yesterday

.gitignore

Remove site from gitignore

last week

.gitmodules

Switch to Terminal theme

last week

README.md

README.md

2 days ago

README

Проектная практика 2026

Участники

ФИО	Учебная группа	Код направления	Профиль
Жилин Константин Владимирович	251-333	09.03.02	Информационные системы и технологии
Беджанов Азамат Асланович	251-333	09.03.02	Информационные системы и технологии

Задание

Базовая часть + вариативная часть (Практическая реализация технологии).

Вариативная часть

Разработка Telegram-бота на Python с использованием библиотеки python-telegram-bot.

Ответственный по практике

Семёнова Валерия Валерьевна.

Проектная деятельность

Практика проводилась в связке с проектом «Развитие дополнительного профессионального образования» по дисциплине «Проектная деятельность».

Воронин Денис Михайлович.

Период проведения

С 03 февраля 2026 г. по 24 мая 2026 г.

Проектная практика 2026. Telegram-бот на Python.

zhilinka.github.io/project-practice-20...

Readme

Activity

0 stars

0 watching

0 forks

Releases

No releases published

Create a new release

Deployments 65

github-pages 36 minutes ago

+ 64 deployments

Packages

No packages published

Publish your first package

Contributors 1

zhilinka

Languages

HTML 71.4%

JavaScript 16.4%

Python 11.8%

CSS 0.4%

Рисунок 2 — Папка docs (документация):

project-practice-2026 / docs /

zhilinka

Add files via upload

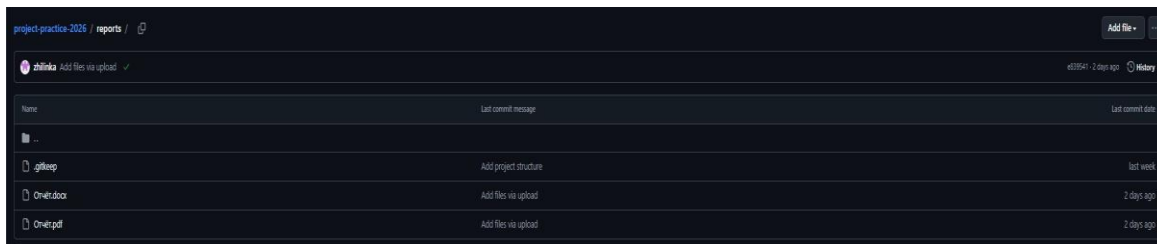
✓

9d08942 · 7 minutes ago

History

Name	Last commit message	Last commit date
...		
.gitkeep	Add project structure	last week
Video.mp4	Add files via upload	7 minutes ago
final-report.md	final-report.md	2 days ago
journal.md	journal.md	4 days ago
partner-report.md	docs: добавлен отчет о взаимодействии с партнером	4 days ago
project.md	project.md	4 days ago
tutorial.md	Add files via upload	4 days ago

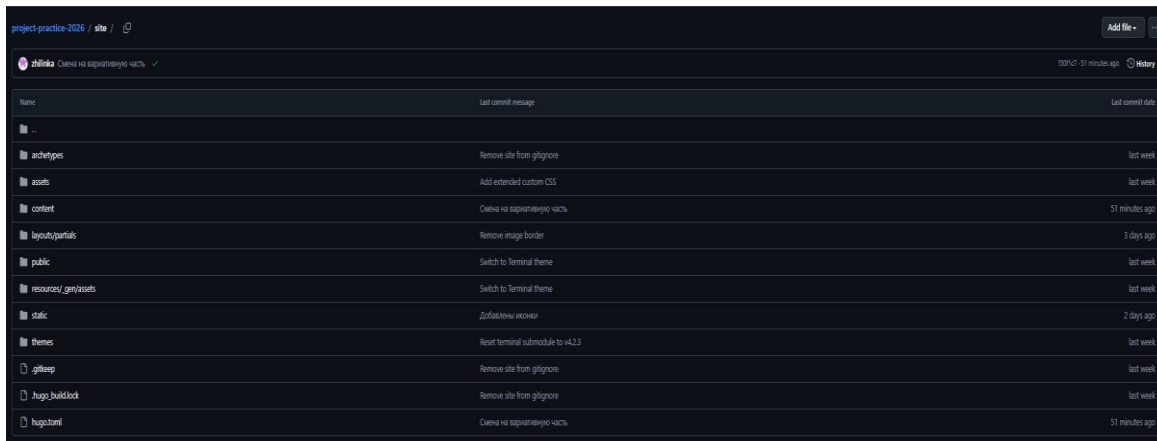
Рисунок 3 — Папка reports (отчёты):



The screenshot shows the 'reports' directory in a project repository. The directory contains a README file and three files: .gitkeep, 0-met.docx, and 0-met.pdf. The commit history for each file is shown in a table below.

Name	Last commit message	Last commit date
..		
.gitkeep	Add project structure	last week
0-met.docx	Add files via upload	2 days ago
0-met.pdf	Add files via upload	2 days ago

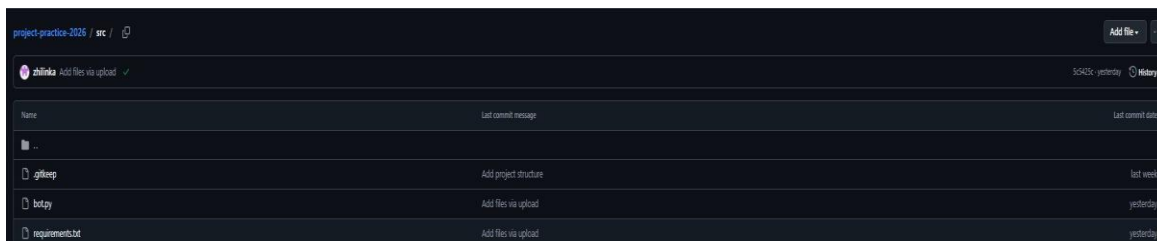
Рисунок 4 — Папка site (Hugo-сайт):



The screenshot shows the 'site' directory in a project repository. The directory contains a README file and several subdirectories and files. The commit history for each file is shown in a table below.

Name	Last commit message	Last commit date
..		
archetypes	Remove site from ghignore	last week
assets	Add extended custom CSS	last week
content	Смена на собственную часть	51 minutes ago
layouts/partials	Remove image border	3 days ago
public	Switch to Terminal theme	last week
resources/_gen/assets	Switch to Terminal theme	last week
static	Добавлены иконки	2 days ago
themes	Reset terminal submodule to v4.2.3	last week
.gitkeep	Remove site from ghignore	last week
.hugo_build.lock	Remove site from ghignore	last week
hugobonf	Смена на собственную часть	51 minutes ago

Рисунок 5 — Папка src (код бота):



The screenshot shows the 'src' directory in a project repository. The directory contains a README file and three files: .gitkeep, bot.py, and requirements.txt. The commit history for each file is shown in a table below.

Name	Last commit message	Last commit date
..		
.gitkeep	Add project structure	last week
bot.py	Add files via upload	yesterday
requirements.txt	Add files via upload	yesterday

2. Документация в Markdown

Подготовлены документы: README.md, описание проекта, журнал прогресса, отчёт о взаимодействии с партнёром, финальный отчёт, tutorial по созданию бота.

3. Статический веб-сайт

Создан статический сайт на Hugo с темой Terminal, опубликован на GitHub Pages.

Рисунок 6 — Главная страница сайта:

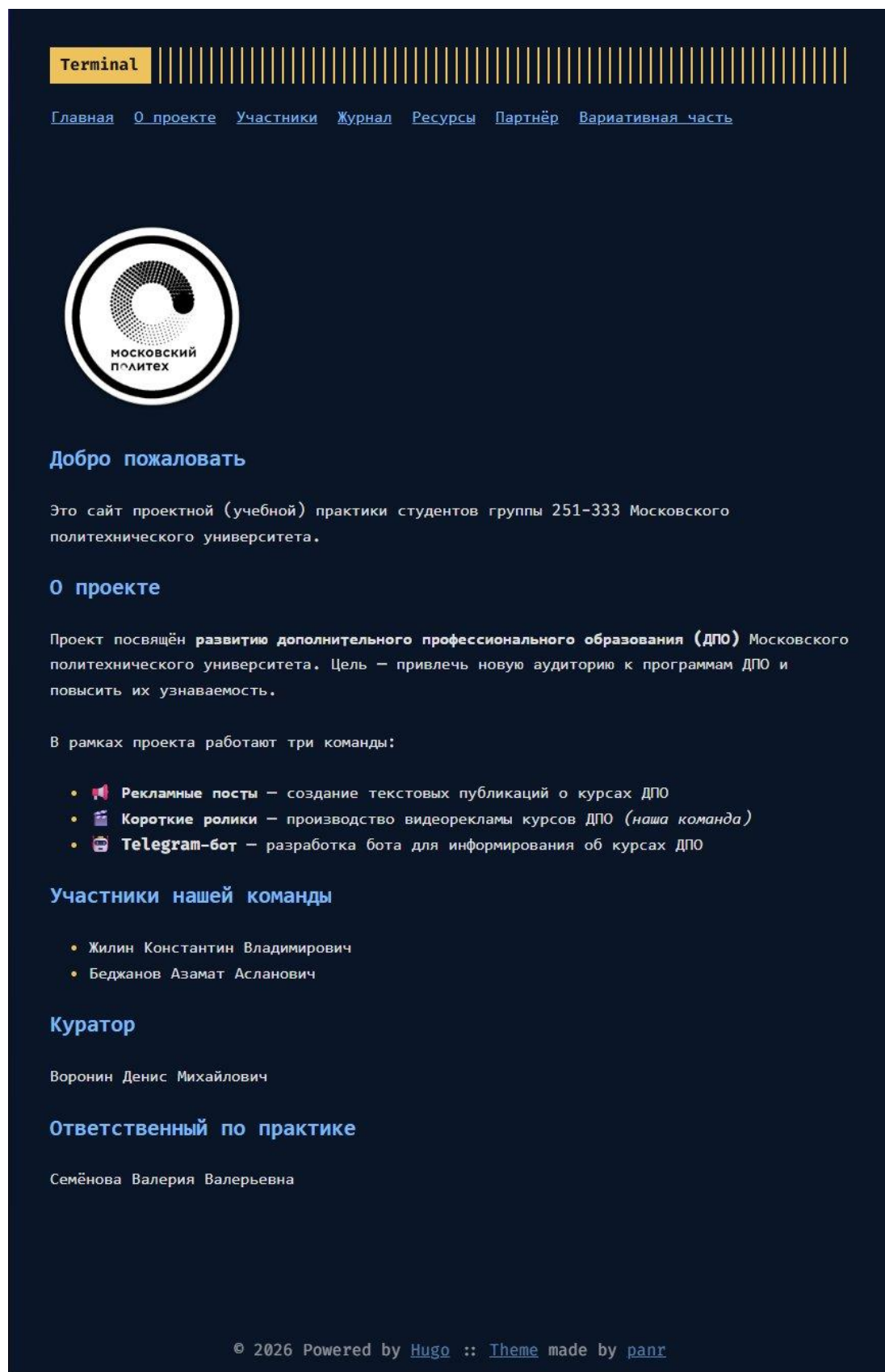


Рисунок 7 — Страница «О проекте»:

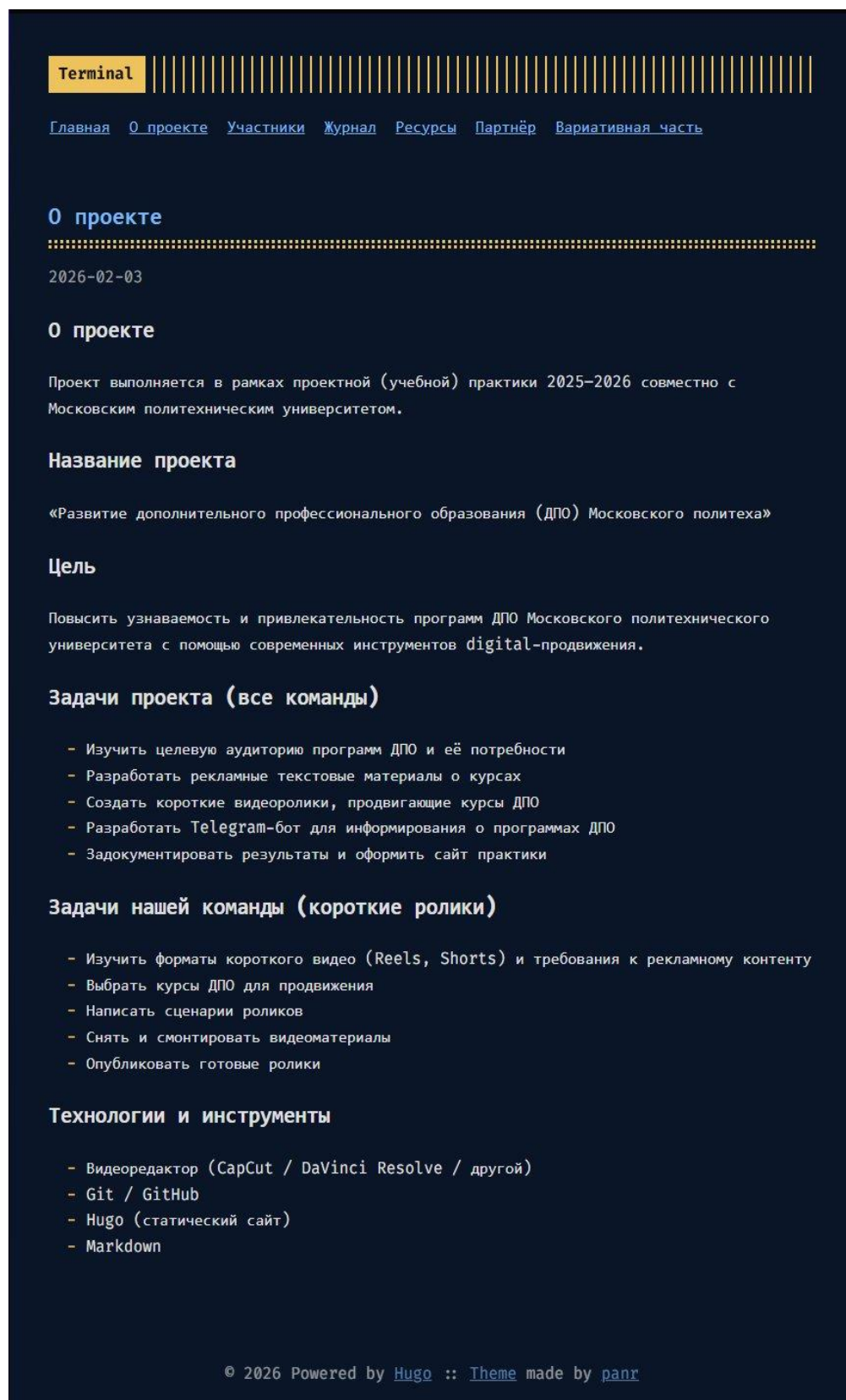


Рисунок 8 — Страница «Участники»:

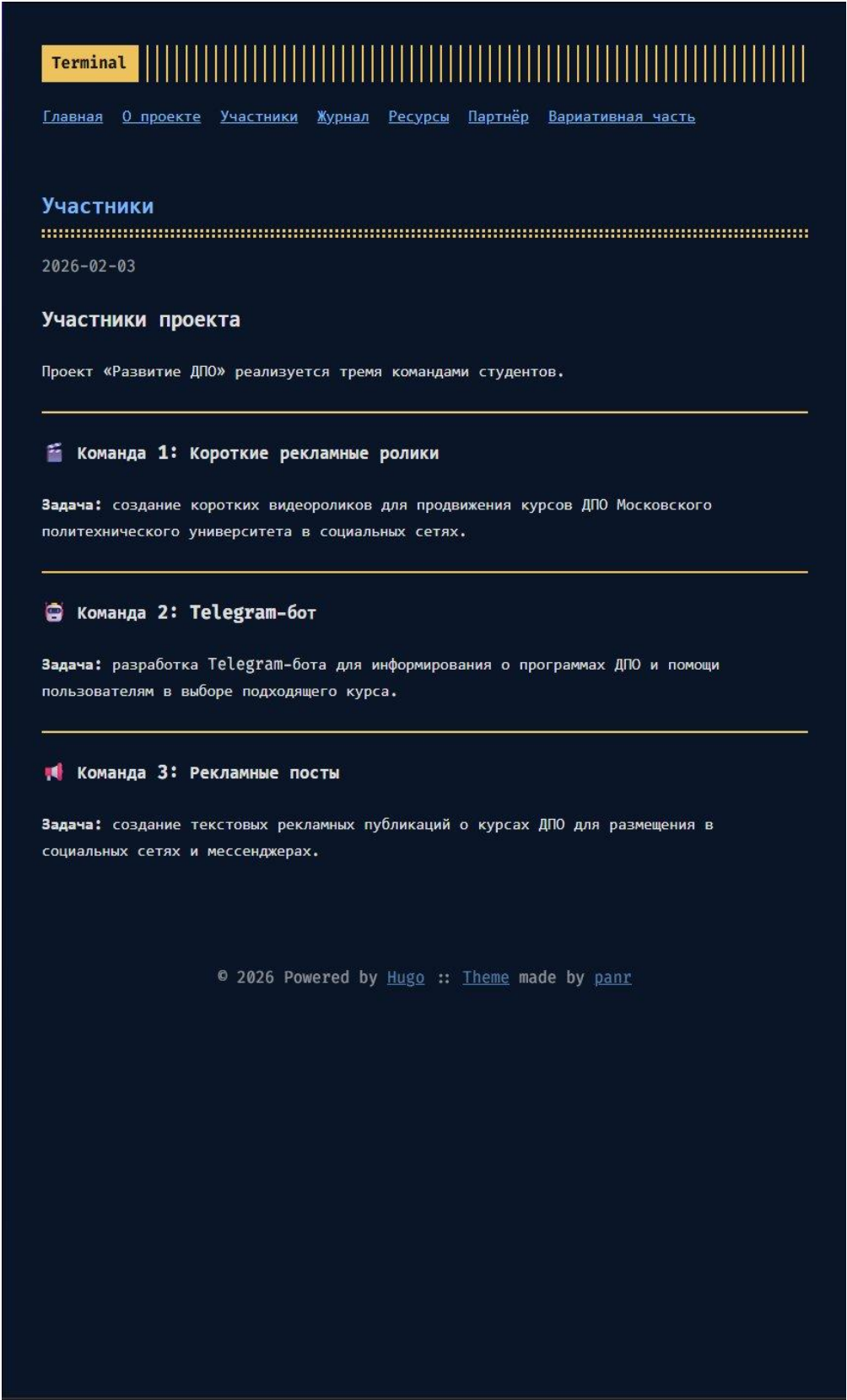


Рисунок 9 — Страница «Журнал»:

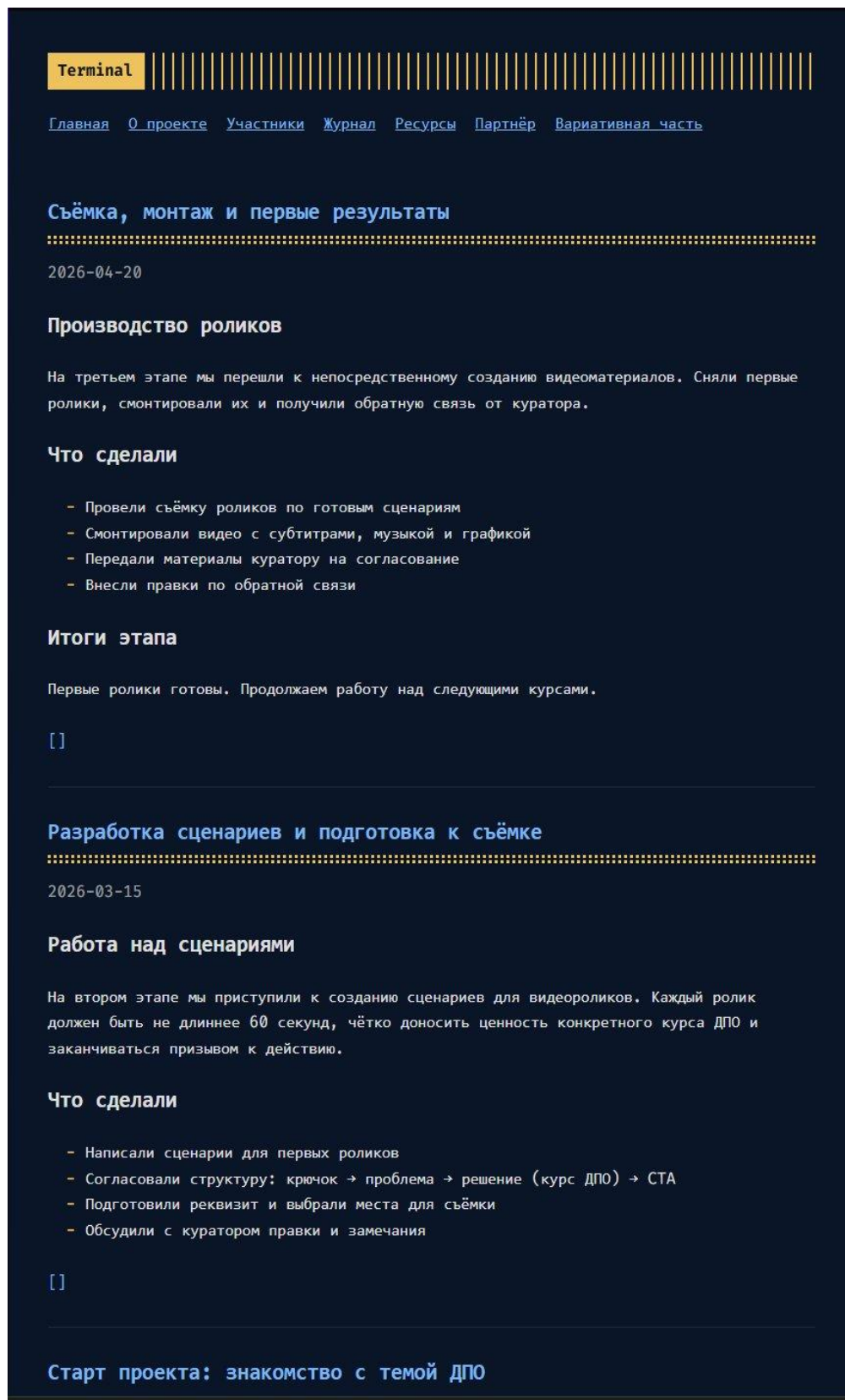
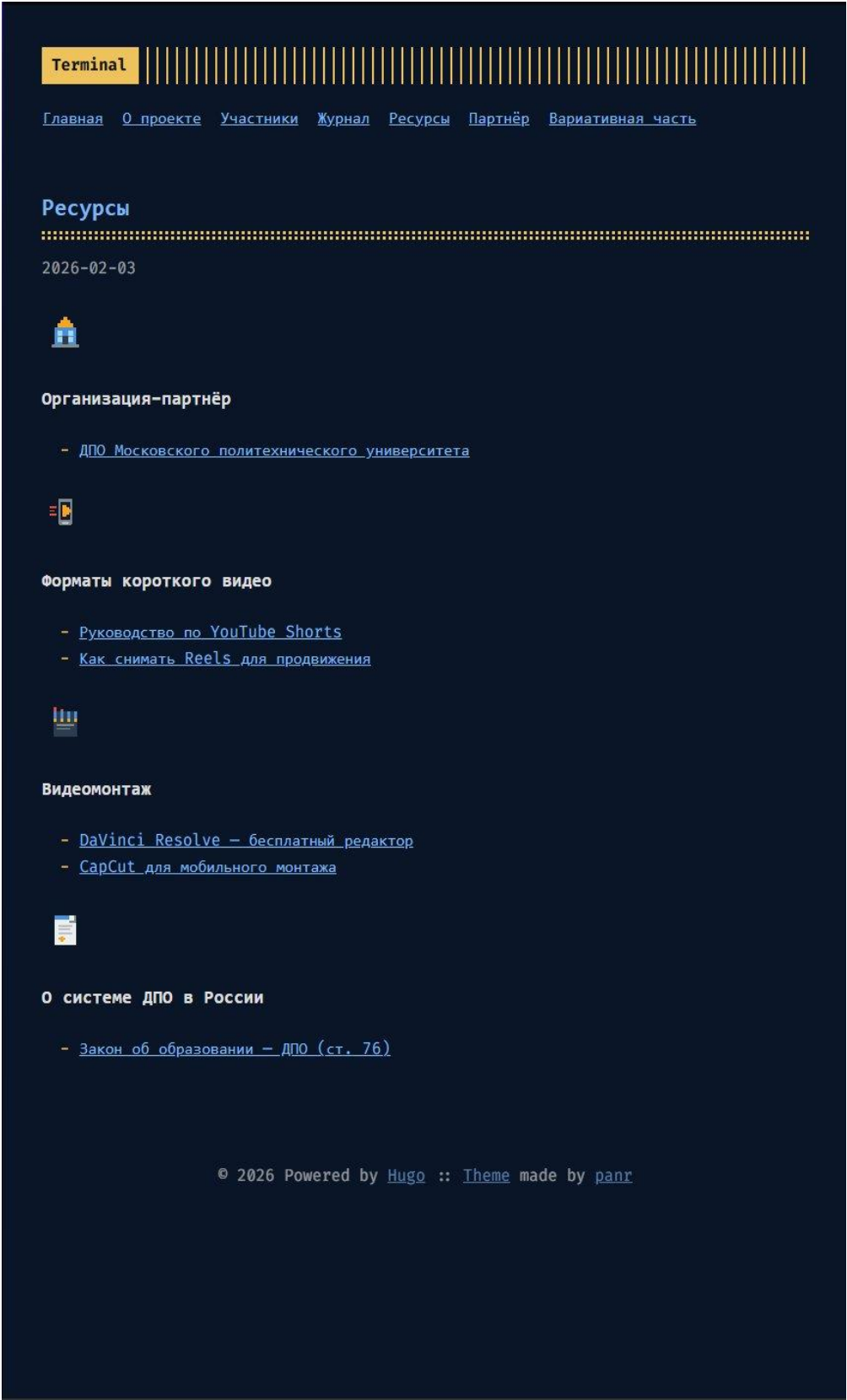


Рисунок 10 — Страница «Ресурсы»:



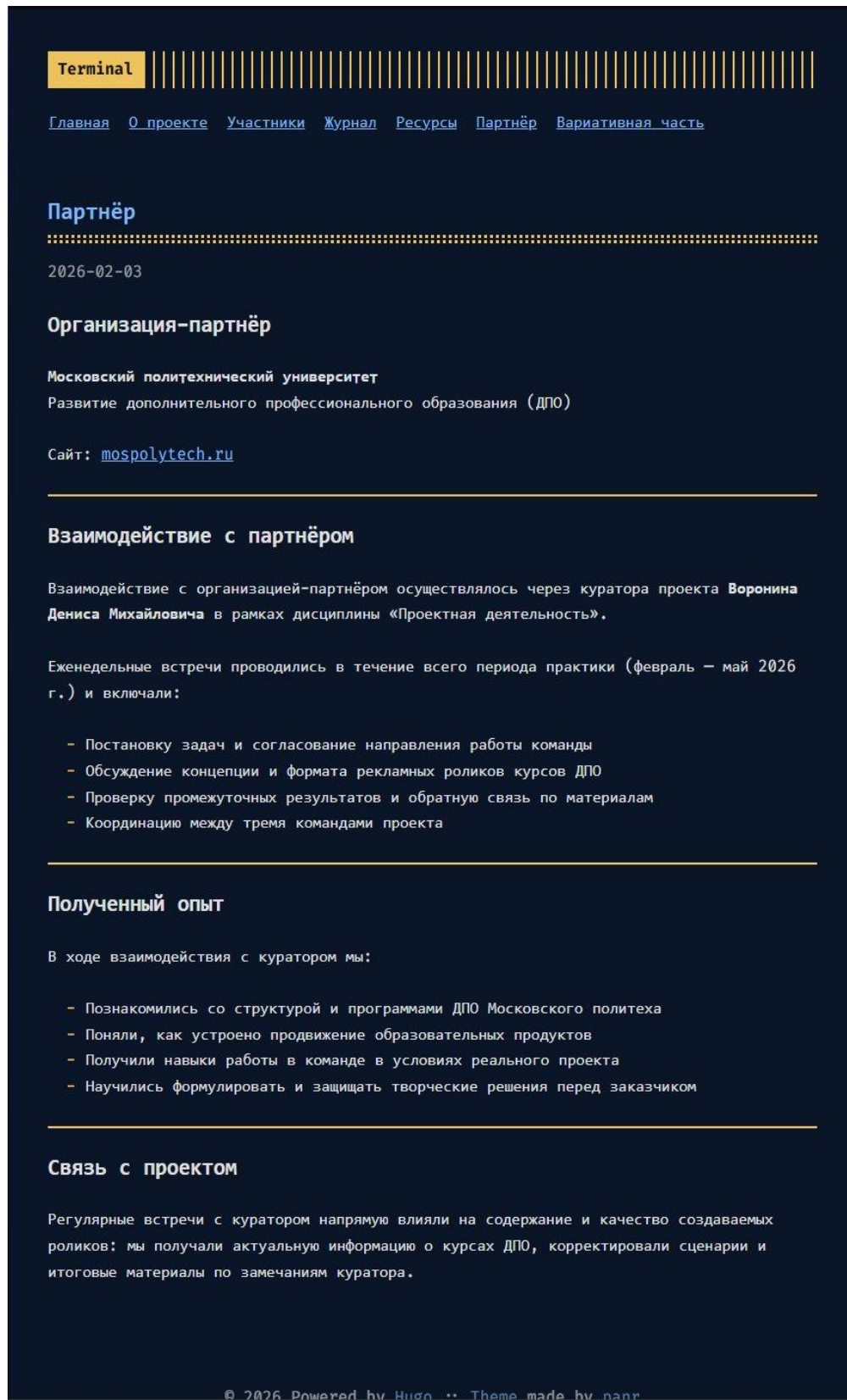


Рисунок 12 — Страница «Вариативная часть»:

Terminal

[Главная](#) [О проекте](#) [Участники](#) [Журнал](#) [Ресурсы](#) [Партнёр](#) [Вариативная часть](#)

Вариативная часть: Telegram-бот для конвертации валют

2026-05-12

 **Currency Bot – Telegram-бот для конвертации валют**

> Учебный проект в рамках проектной практики

Московский политехнический университет, кафедра информационных технологий

Группа 251-333 | Направление: 09.03.02 Информационные системы и технологии

 **Описание**

Currency Bot (`@mospolytech_currency_bot`) – это Telegram-бот на Python, который позволяет получать актуальные курсы валют и конвертировать суммы в режиме реального времени через внешний API.

Что умеет бот:

-  Показывает актуальный курс любой из 8 поддерживаемых валют
-  Конвертирует сумму одной командой `/rate`
-  Пошаговая конвертация через inline-кнопки `/convert`
-  Отмена текущего диалога командой `/cancel`
-  Корректно обрабатывает ошибки и недоступность API

 **Стек технологий**

ТЕХНОЛОГИЯ	НАЗНАЧЕНИЕ
Python 3.10+	Язык программирования
python-telegram-bot 22.x	Асинхронная работа с Telegram Bot API
requests	HTTP-запросы к API курсов валют
open.er-api.com	Источник актуальных курсов валют
asyncio	Асинхронное выполнение запросов
Git / GitHub	Контроль версий

 **Поддерживаемые валюты**

4. Взаимодействие с партнёром

Взаимодействие осуществлялось через куратора проекта Воронина Д.М. в формате еженедельных встреч. Написан отчёт в формате Markdown.

4.2 Вариативная часть — Telegram-бот

Разработан Telegram-бот `@mospolytech_currency_bot` для конвертации валют. Бот написан на Python с библиотекой `python-telegram-bot 22.x` и получает актуальные курсы через API `open.er-api.com`.

Поддерживаемые валюты: USD, EUR, RUB, GBP, CNY, JPY, CHF, TRY

Команды бота:

- `/start` — приветствие и список команд
- `/help` — подробная инструкция
- `/rate <ИЗ> <В> [сумма]` — курс или конвертация
- `/convert` — пошаговая конвертация через inline-кнопки
- `/cancel` — отмена текущего диалога

Рисунок 13 — Меню команд бота в Telegram:

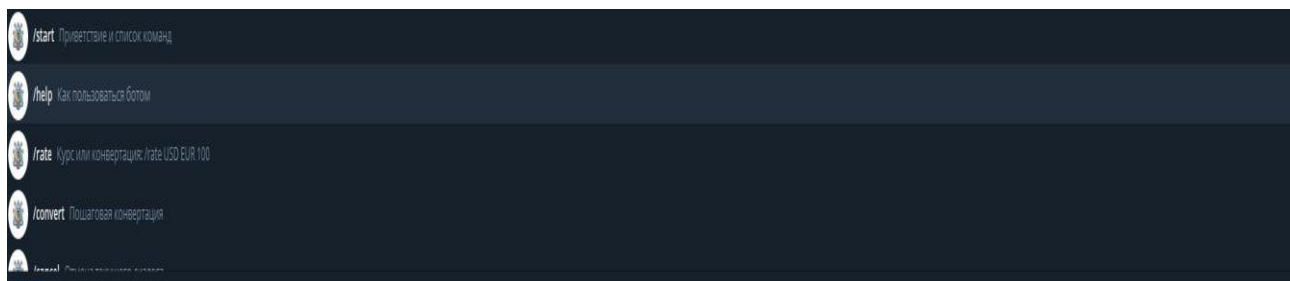
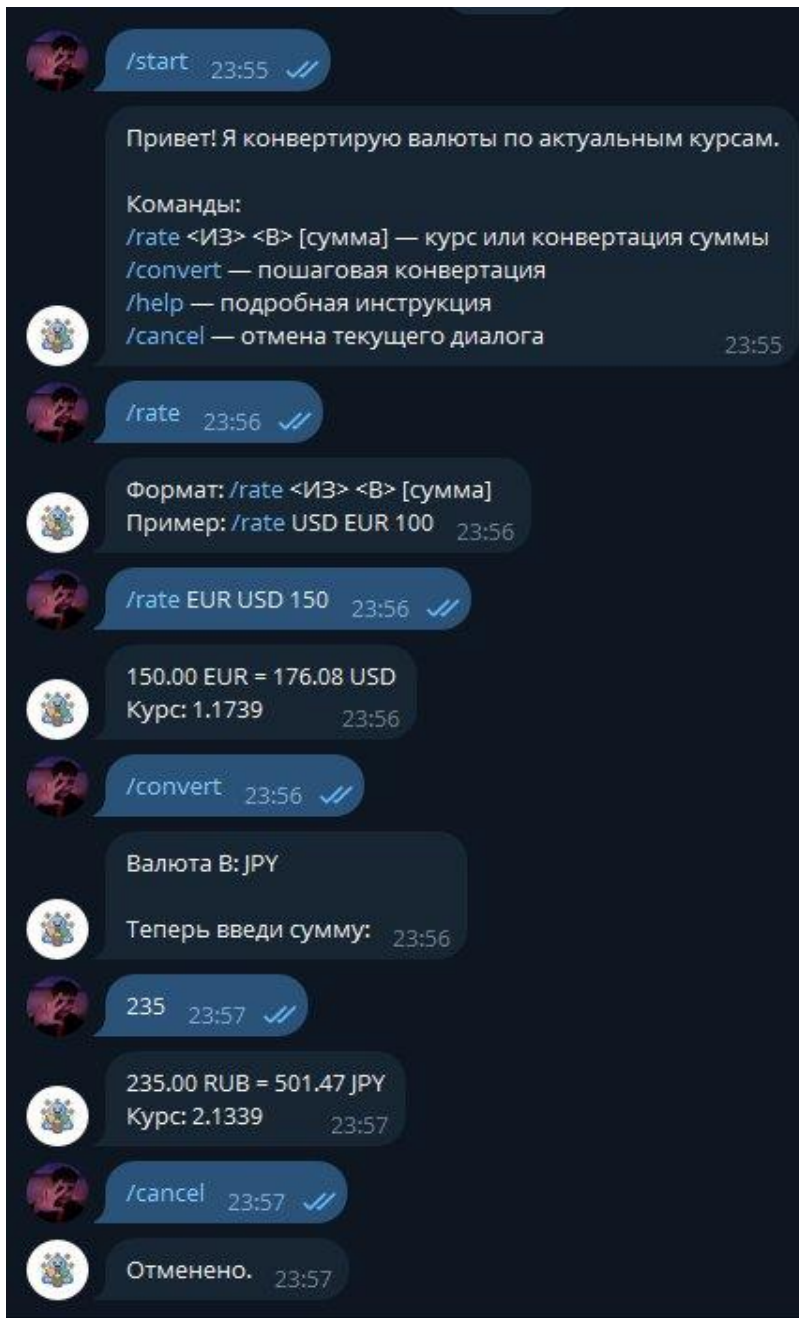


Рисунок 14 — Демонстрация работы бота (/start, /rate, /convert, /cancel):



Листинг 1 — Константы и состояния диалога:

```
API_URL = "https://open.er-api.com/v6/latest/{base}"
TIMEOUT_S = 5

CURRENCIES = ["USD", "EUR", "RUB", "GBP", "CNY", "JPY", "CHF", "TRY"]
BUTTONS_PER_ROW = 4

SELECT_FROM = 0
SELECT_TO = 1
ENTER_AMOUNT = 2
```

Листинг 2 — Загрузка токена из .env:

```
def load_env_file(path: str = ".env") -> None:
    if not os.path.exists(path):
        return
    with open(path, "r", encoding="utf-8") as f:
        for raw_line in f:
            line = raw_line.strip()
            if not line or line.startswith("#") or "=" not in line:
                continue
            key, value = line.split("=", 1)
            key = key.strip()
            value = value.strip().strip('"').strip("'")
            if key and key not in os.environ and value:
                os.environ[key] = value
```

Листинг 3 — Клавиатура выбора валюты:

```
def build_currency_keyboard(
    *, exclude: Optional[str] = None, prefix: str
) -> InlineKeyboardMarkup:
    items = [c for c in CURRENCIES if c != exclude]
    rows: list[list[InlineKeyboardButton]] = []
    for i in range(0, len(items), BUTTONS_PER_ROW):
        row = [
            InlineKeyboardButton(
                text=code, callback_data=f"{prefix}:{code}"
            )
            for code in items[i : i + BUTTONS_PER_ROW]
        ]
        rows.append(row)
    return InlineKeyboardMarkup(rows)
```

Листинг 4 — Получение курсов валют через API:

```
async def fetch_rates(base: str) -> Optional[dict]:
    url = API_URL.format(base=base)

    def _do_request() -> dict:
        return requests.get(url, timeout=TIMEOUT_S).json()

    try:
        data = await asyncio.to_thread(_do_request)
    except Exception:
        logging.exception("API request failed: %s", url)
        return None

    if not isinstance(data, dict):
        return None
    if data.get("result") != "success":
        return None

    rates = data.get("rates")
    if not isinstance(rates, dict):
        return None

    return rates

async def get_rate(from_code: str, to_code: str) -> Optional[float]:
    rates = await fetch_rates(from_code)
    if not rates:
        return None
```

```

raw = rates.get(to_code)
try:
    return float(raw)
except (TypeError, ValueError):
    return None

```

Листинг 5 — Команды /start и /help:

```

async def start_cmd(
    update: Update, context: ContextTypes.DEFAULT_TYPE
) -> None:
    text = (
        "Привет! Я конвертирую валюты по актуальным курсам.\n\n"
        "Команды:\n"
        "/rate <ИЗ> <В> [сумма] — курс или конвертация суммы\n"
        "/convert — пошаговая конвертация\n"
        "/help — подробная инструкция\n"
        "/cancel — отмена текущего диалога"
    )
    await update.effective_message.reply_text(text)

async def help_cmd(
    update: Update, context: ContextTypes.DEFAULT_TYPE
) -> None:
    text = (
        "Как пользоваться:\n\n"
        "/rate <ИЗ> <В> [сумма]\n"
        "Примеры:\n"
        "- /rate USD EUR\n"
        "- /rate USD RUB 100\n\n"
        "/convert\n"
        "- выбери валюту ИЗ\n"
        "- выбери валюту В\n"
        "- введи сумму обычным текстом\n\n"
        f"Поддерживаемые валюты: {' '.join(CURRENCIES)}"
    )
    await update.effective_message.reply_text(text)

```

Листинг 6 — Команда /rate:

```

async def rate_cmd(
    update: Update, context: ContextTypes.DEFAULT_TYPE
) -> None:
    msg = update.effective_message
    args = context.args
    if len(args) < 2 or len(args) > 3:
        await msg.reply_text(
            "Формат: /rate <ИЗ> <В> [сумма]\nПример: /rate USD EUR 100"
        )
        return

    from_code = args[0].upper()
    to_code = args[1].upper()
    if from_code not in CURRENCIES or to_code not in CURRENCIES:
        await msg.reply_text(
            f"Неподдерживаемая валюта. Доступны: {' '.join(CURRENCIES)}"
        )
        return
    if from_code == to_code:
        await msg.reply_text("Валюты «ИЗ» и «В» должны отличаться.")
        return

    amount: Optional[float] = None

```

```

if len(args) == 3:
    try:
        amount = float(args[2].replace(",", ""))
    except ValueError:
        await msg.reply_text("Сумма должна быть числом, например 100")
        return
    if amount <= 0:
        await msg.reply_text("Сумма должна быть больше 0.")
        return

rate = await get_rate(from_code, to_code)
if rate is None:
    await msg.reply_text(
        f"Не удалось получить курс {from_code} → {to_code}."
    )
    return

if amount is None:
    await msg.reply_text(
        f"1 {from_code} = {format_rate(rate)} {to_code}"
    )
else:
    result = amount * rate
    await msg.reply_text(
        f"{format_amount(amount)} {from_code} = "
        f"{format_amount(result)} {to_code}\n"
        f"Курс: {format_rate(rate)}"
    )

```

Листинг 7 — Диалог /convert (все шаги) :

```

async def convert_entry(
    update: Update, context: ContextTypes.DEFAULT_TYPE
) -> int:
    await update.effective_message.reply_text(
        "Выбери валюту, ИЗ которой конвертируем:",
        reply_markup=build_currency_keyboard(prefix="FROM"),
    )
    return SELECT_FROM

async def convert_select_from(
    update: Update, context: ContextTypes.DEFAULT_TYPE
) -> int:
    query = update.callback_query
    await query.answer()
    prefix, code = query.data.split(":", 1)
    context.user_data["from"] = code
    await query.edit_message_text(
        f"Валюта ИЗ: {code}\n\nТеперь выбери валюту, В которую:",
        reply_markup=build_currency_keyboard(exclude=code, prefix="TO"),
    )
    return SELECT_TO

async def convert_select_to(
    update: Update, context: ContextTypes.DEFAULT_TYPE
) -> int:
    query = update.callback_query
    await query.answer()
    prefix, code = query.data.split(":", 1)
    context.user_data["to"] = code
    await query.edit_message_text(
        f"Валюта В: {code}\n\nТеперь введи сумму:"
    )

```

```

    )
    return ENTER_AMOUNT

async def convert_enter_amount(
    update: Update, context: ContextTypes.DEFAULT_TYPE
) -> int:
    msg = update.effective_message
    text = (msg.text or "").strip()
    try:
        amount = float(text.replace(",", "", ""))
    except ValueError:
        await msg.reply_text("Введи число (например 100 или 12.5):")
        return ENTER_AMOUNT

    if amount <= 0:
        await msg.reply_text("Сумма должна быть больше 0:")
        return ENTER_AMOUNT

    from_code = context.user_data.get("from")
    to_code = context.user_data.get("to")

    rate = await get_rate(from_code, to_code)
    if rate is None:
        await msg.reply_text(
            f"Не удалось получить курс {from_code} → {to_code}."
        )
        return ConversationHandler.END

    result = amount * rate
    await msg.reply_text(
        f"{format_amount(amount)} {from_code} = "
        f"{format_amount(result)} {to_code}\n"
        f"Курс: {format_rate(rate)}"
    )
    context.user_data.pop("from", None)
    context.user_data.pop("to", None)
    return ConversationHandler.END

```

Листинг 8 — Сборка и запуск приложения:

```

def build_app(token: str) -> Application:
    request = HTTPXRequest(
        connect_timeout=20, read_timeout=20,
        write_timeout=20, pool_timeout=20
    )
    app = (
        Application.builder()
        .token(token)
        .request(request)
        .post_init(post_init)
        .build()
    )

    app.add_handler(CommandHandler("start", start_cmd))
    app.add_handler(CommandHandler("help", help_cmd))
    app.add_handler(CommandHandler("rate", rate_cmd))

    conv = ConversationHandler(
        entry_points=[CommandHandler("convert", convert_entry)],
        states={
            SELECT_FROM: [CallbackQueryHandler(
                convert_select_from, pattern=r"^FROM:")],
            SELECT_TO: [CallbackQueryHandler(

```

```

        convert_select_to, pattern=r"^TO:"],
        ENTER_AMOUNT:[MessageHandler(
            filters.TEXT & ~filters.COMMAND,
            convert_enter_amount)],
    },
    fallbacks=[CommandHandler("cancel", cancel_cmd)],
    allow_reentry=True,
)
app.add_handler(conv)
app.add_handler(CommandHandler("cancel", cancel_cmd))
return app

def main() -> None:
    if not os.getenv("BOT_TOKEN"):
        load_env_file(".env")
    token = os.getenv("BOT_TOKEN")
    if not token:
        raise RuntimeError("BOT_TOKEN is not set.")
    build_app(token).run_polling(bootstrap_retries=3)

if __name__ == "__main__":
    main()

```

ЗАКЛЮЧЕНИЕ

В ходе проектной практики студенты успешно выполнили все задачи базовой и вариативной частей.

В рамках базовой части были освоены навыки работы с Git/GitHub, создания статических сайтов на Hugo, написания технической документации в формате Markdown, а также организовано взаимодействие с организацией-партнёром через куратора проекта.

В рамках вариативной части разработан Telegram-бот для конвертации валют с поддержкой 8 валют, двумя режимами работы (/rate и /convert) и корректной обработкой ошибок.

Ссылки на проект:

- Репозиторий: <https://github.com/zhilinka/project-practice-2026>
- Сайт: <https://zhilinka.github.io/project-practice-2026/>
- Тг-бот: @mospolytech_currency_bot

СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ

1. ДПО Московского политехнического университета. — URL: <https://mospolytech.ru/dopolnitelnoe-professionalnoe-obrazovanie/>
2. Hugo. Официальная документация. — URL: <https://gohugo.io/documentation/>
3. Git. Официальная документация. — URL: <https://git-scm.com/book/ru/v2>
4. python-telegram-bot. Документация. — URL: <https://python-telegram-bot.org>
5. ExchangeRate API. — URL: <https://open.er-api.com>
6. Markdown. Руководство по синтаксису. — URL: <https://www.markdownguide.org>